Análisis Técnico de Base de Datos y APIs

"La prisa es enemiga de un estudio"

Respuesta a los criterios de evaluación de la clínica

Autor: Alejandro Pavón

Dada: Neon Nebula Studio - Madrid,España - Portugalete

Creña adaptativa: PGE-2026 - Marzo 2026

1. Justificación del Motor de Base de Datos

1.1 Elección Vigente: PostgreSQL 16+

PostgreSQL fue seleccionado como SGBDR para Mears. Esta elección se fundamenta en las necesidades del proyecto y las ventajas que PostgreSQL ofrece en este contexto.

1.2 Advocación al caso de uso

Mears es una aplicación con un modelo de datos relacional complejo (usuarios que poseen prendas, que se agrupan en pedidos, que generan facturas, que requieren pagos). PostgreSQL ofrece un soporte robusto para transacciones ACID, integridad referencial y tipos de datos avanzados (JSONB, arrays) que son útiles para las reacciones fundamentales (ej. con megafonía referencia).

1.3 Ventajas específicas sobre alternativas

Característica	Ventaja sobre MySQL	Ventaja sobre SQLite
Arrays nativos (TEXT[], INTEGER[])	Evita tablas pivote (N:M) para almacenar multivalores (ej. tallas)	Idem
JSONB binario (json_data_gin_idx, jsonb_path_ops)	Consultas rápidas sobre datos semi-estructurados (ej. configuraciones de usuario)	Parcial
Integridad referencial (FK) y transacciones ACID	Garantías más fuertes para la consistencia de datos	Parcial
Constraints avanzados (CHECK, UNIQUE WHERE)	Permite reglas de negocio más expresivas (ej. stock_quantity >= 0)	Parcial
Vistas (views)	Simplifica consultas complejas y mejora la mantenibilidad	Parcial
Índices parciales (WHERE condición)	Optimización de consultas específicas (ej. idx_users_username WHERE NOT NULL)	✗
Escalabilidad en producción (Railway)	Soporta múltiples conexiones concurrentes	✗
Extensión y open source	Ecosistema rico (PostGIS, pg_trgm, etc.)	Parcial

1.4 Funcionalidades aprovechadas

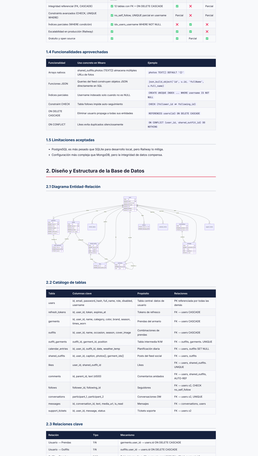
Característica	Uso en Mears	Ejemplo
Arrays nativos	Almacenar tallas disponibles (TEXT[]) sin crear tabla pivote	sizes TEXT[] DEFAULT '{}'
Funciones JSON	Guardar datos flexibles como preferencias (JSON) sin alterar el esquema	json_build_object('id', u.id, 'firstName', u.first_name)
Constraints avanzados	Garantizar tallas válidas y stock no negativo	CHECK (NEXISTS (SELECT * FROM UNNEST(sizes) WHERE NOT IN ('S', 'M', 'L', 'XL')) AND stock_quantity >= 0)
Constraint CHECK	Eliminar usuarios propios o roles no admitidos	DELETE FROM users WHERE id = $1 AND id <> $2 AND role IN ('admin', 'moderator')
ON CONFLICT	Lista wishlist duplicada silenciosamente	INSERT INTO wishlist (user_id, product_id) VALUES ($1, $2) ON CONFLICT DO NOTHING

1.5 Limitaciones aceptadas

- PostgreSQL no más rápido para operaciones de lectura simples que SQLite
- Configuración más compleja (pg.config.ini) que la integración de datos comprimidos

2. Diseño y Estructura de la Base de Datos

2.1 Diagrama Entidad-Relación



2.2 Catálogo de tablas

Tabla	Columnas clave	Propósito
users	id (PK), email, hashed_password, first_name, last_name, phone, gender, birthdate, role, stock_quantity, is_active, is_verified, created_at, updated_at, last_login	Perfil de usuario, datos de contacto, roles, estado de cuenta
refresh_tokens	id, user_id, token_hashed, expiresAt	Tokens de refresco
garments	id, user_id, name, category, color, brand, sizes, image_urls, is_active, created_at	Prendas del armario
outfits	id, user_id, name, occasions, season, cover_image, created_at	Combinaciones de prendas
outfit_items	outfit_id, garment_id, position	Tabla intermedia N:M
calendar_entries	id, user_id, outfit_id, date, weather_temp	Planificación diaria
shared_outfits	id, user_id, outfit_id, expiresAt, permanent_link	Pases del fondo actual
likes	user_id, shared_outfit_id	Likes
comments	id, parent_id, text, created_at	Comentarios anidados
follows	follower_id, following_id	Seguidores
conversations	participant_1, participant_2	Conversaciones DM
messages	id, conversation_id, text, media_url, is_read	Mensajes
support_tickets	id, user_id, message, status	Ticket soporte

2.3 Relaciones clave

Relación	Tipo	Descripción
Usuario → Prendas	1:N	garments.user_id → users.id ON DELETE CASCADE
Usuario → Outfits	1:N	outfits.user_id → users.id ON DELETE CASCADE
Outfit → Prendas	N:M	Tabla intermedia outfit_garments con UNIQUE(outfit_id, garment_id)
Post → Likes	1:N	UNIQUE(user_id, shared_outfit_id) evita duplicados
Comentarios → Comentarios	Auto-referencial	comments.parent_id → comments.id (respuestas anidadas)
Seguidores	Auto-referencial	follows.follower_id → follows.following_id
Conversación → Mensajes	1:N	messages.conversation_id → conversations.id

2.4 Normalización

Array/Texto	Tabla relacionada	Justificación
sizes	Catálogo administrable	Evita errores por tallas inválidas como en PostgreSQL
gender	Catálogo administrable	Evita inconsistencias (ej. "masculino" vs "hombre")
sizes	Sin dependencias transitivas	Tabla del usuario sin dependencias ni JSON, ya en equilibrio

2.5 Índices de rendimiento

Índice	Tabla / Columnas	Qué optimiza
idx_users_email	users(email)	Login y búsqueda de email
idx_refresh_tokens_token_hashed WHERE NOT NULL	refresh_tokens(token_hashed)	Búsqueda por campo hashed
idx_garments_user	garments(user_id)	Listar prendas del usuario
idx_garments_category	garments(user_id, category)	Filtrar por categoría
idx_calendar_entries_user_date	calendar_entries(user_id, date)	Vista mensual por fecha
idx_calendar_user_date	calendar_permanent_id, date)	Calendario mensual
idx_feed_recent	feed(created_at DESC)	Orden feed
idx_comments_post	comments(shared_outfit_id)	Comentarios de un post
idx_messages_conversation	messages(conversation_id, created_at)	Historial de chat
idx_follows_follower	follows(follower_id)	Seguidores / Siguiendo

2.6 Tipos de datos

Tipo	Uso	Justificación
DATE	date	Sin componente hora
TIMESTAMPTZ	created_at, updated_at, expiresAt	Con zona horaria
TEXT	text, caption, password_hash	Contenido libre
BOOLEAN	is_active, is_favorite, is_public	Flags con DEFAULT false
TIMESTAMPTZ WITH TIME ZONE	created_at, expires_at	Fechas con zona horaria
TEXT[], INTEGER[]	sizes, garment_ids, type_preferences	Arrays nativos PostgreSQL
INTEGER	weather_temp	Temperatura con decimales

3. Conexión de la Aplicación con la BBDD

3.1 Módulo: Pool de conexiones con pg

```
const { Pool, Client } = require('pg');
const dotenv = require('dotenv').config({ path: '.env.local' });
const pool = new Pool(config);
pool.on('error', (err, client) => { console.error('Error inesperado en el pool de conexiones', err); });
module.exports = { query: (text, params) => pool.query(text, params), pool, };
```

3.2 Configuración dual: Local y Producción

```
const pool_config = process.env.DATABASE_URL
  ? { connectionString: process.env.DATABASE_URL,
      ssl: { rejectUnauthorized: false },
      max: 20, idleTimeoutMillis: 30000, connectionTimeoutMillis: 2000 }
  : { host: process.env.DB_HOST || 'localhost',
      port: parseInt(process.env.DB_PORT || '5432'),
      database: process.env.DB_NAME || 'mears_dev',
      user: process.env.DB_USER || process.env.USER,
      password: process.env.DB_PASSWORD || undefined,
      max: 20, idleTimeoutMillis: 30000, connectionTimeoutMillis: 2000 };
```

3.3 Diagrama de flujo de conexión

3.4 Buenas prácticas

Práctica	Implementación	Beneficio
Connection Pooling	max: 20 conexiones multiplexadas	Evita abrir/cerrar una nueva cada vez
Timeout inactividad	idleTimeoutMillis: 30000	Libera conexiones ociosas
Timeout conexión	2000ms (local), 10000ms (prod)	Fallo rápido si BD no responde
SSL en producción	ssl: { rejectUnauthorized: false }	Cifra conexiones requeridas BD
Variables de entorno	.env.local, .gitignore	Protege secretos
Fallo silencioso	pool.on('error', (err) => console.error)	Evita crash de BD caída
Queries parametrizadas	$1, $2, $3...	Previene SQL injection

3.5 Consultas parametrizadas

```
// ✅ CORRECTO - parametrizada (SEGURA de ataques SQL)
const POST = async (req) => { const { email, name } = req.body;
  const result = pool.query('SELECT * FROM users WHERE email = $1', [email]);
  return { statusCode: 200, body: JSON.stringify(result.rows[0]) }; };

// ❌ INCORRECTO - concatenación (VULNERABLE a ataques SQL)
const GET = (req, res) => { const id = req.query.id;
  const result = pool.query('SELECT * FROM users WHERE id = ' + id);
  res.send(result.rows[0]); };

// ✅ MÁS SEGURO: para operaciones simples usamos un ORM
const User = require('./models/User');
const GET = (req, res) => { const id = req.query.id;
  const user = User.findById(id, ['id', 'email', 'name']);
  res.send(user); };
```

4. Integración y Uso de APIs

4.1 API REST Propia

Arquitectura MVC con 11 módulos. Todas siguen `/api/v1/<recurso>`.

```
backend/
├── server.js          # Entrada, rutas, CORS
├── db.js              # Pool de conexiones
├── middleware/        # Auth, validación, rate limit
├── routes/            # Definición de endpoints
│   ├── auth.routes.js
│   ├── users.routes.js
│   ├── garments.routes.js
│   ├── outfits.routes.js
│   ├── calendar.routes.js
│   ├── feed.routes.js
│   ├── social.routes.js
│   ├── messages.routes.js
│   ├── notifications.routes.js
│   ├── support.routes.js
│   └── admin.routes.js
└── controllers/       # Lógica de negocio
    ├── auth.controller.js
    ├── users.controller.js
    ├── garments.controller.js
    ├── outfits.controller.js
    ├── calendar.controller.js
    ├── feed.controller.js
    ├── social.controller.js
    ├── messages.controller.js
    ├── notifications.controller.js
    ├── support.controller.js
    └── admin.controller.js
```

Tabla completa de endpoints

Módulo	Método	Ruta	Descripción	Auth
Auth	POST	/api/v1/auth/signup	Registro con validación	No
Auth	POST	/api/v1/auth/login	Login, genera access+refresh	No
Auth	POST	/api/v1/auth/refresh	Renueva access token	No
Auth	POST	/api/v1/auth/logout	Invalida refresh token	Sí
Garments	GET	/api/v1/garments	Lista Prendas (category, color)	Sí
Garments	POST	/api/v1/garments	Crear prenda + images	Sí
Garments	PUT	/api/v1/garments/:id	Actualizar prenda	Sí
Garments	DELETE	/api/v1/garments/:id	Eliminar prenda + cleanup	Sí
Garments	GET	/api/v1/garments/latest-color	Detectar color CrossTheme	Sí
Outfits	GET/POST/PUT/DEL	/api/v1/outfits	CRUD de outfits	Sí
Calendar	GET/POST/PUT/DEL	/api/v1/calendar	Planificación de looks	Sí
Social	GET	/api/v1/feed/feed	Feed (follows/following)	Sí
Social	POST	/api/v1/follows	Publicar con Twitter Filter	Sí
Social	PATCH/DEL	/api/v1/follows/:id	Editar / Eliminar	Sí
Social	GET/POST/DEL	/api/v1/social/shared/:id/comments	Comentarios anidados	Sí
Social	POST	/api/v1/social/shared/:id/like	Alternar like	Sí
Social	POST	/api/v1/social/shared/:id/save	Guardar / quitar clima	Sí
Social	POST	/api/v1/social/shared/:id/wear	Clon a outfits propios	Sí
Social	POST	/api/v1/social/mapping	Recomendaciones compra	Sí
Social	GET	/api/v1/social/search/users	Buscar usuarios	Sí
Messages	GET/POST	/api/v1/messages/conversations	Mensajería directa	Sí
Profile	GET/PUT	/api/v1/profile/me	Perfilado de usuario	Sí
Stats	GET	/api/v1/stats	Estadísticas del armario	Sí
Admin	GET/PUT/DEL	/api/v1/admin/users	Panel de administración	Admin

4.2 APIs Externas Integradas

API	Propósito	Autenticación	Límites
Open-Meteo (v1/forecast)	Clima actual/horario para recomendaciones	Ninguna clave	10000/día, 600/min
Nominatim/Open Street Map	Reverse geocoding (lat,lon → ciudad)	Header User-Agent obligatorio	1 req/seg
Open Meteo	Clima actual y pronóstico 5 días	Sin API key (gratuito)	weather service
Nominatim (OSM)	Geocoding (ciudad → coordenadas)	Sin key. Cache 24h obligatorio	weather service
Stripe (futuro)	Integración cobros de productos	Header: Y-API-KEY	ai service
Cloudinary	Transcripción audio → texto	Basic + Cloudinary/Stripe	ai service
CrossTheme	Detección color dominante	Local (sin API)	garments controller.js

4.3 Patrón de resilencia: Fallback

```
// OPEN-METEO: Sin API key, completamente gratuito
const fetchWeather = (lat, lon) => {
  const URL = `https://api.open-meteo.com/v1/forecast?latitude=${lat}&longitude=${lon}&current_weather=true&hourly=temperature_2m,relative_humidity_2m,weather_code&timezone=auto&forecast_days=1`;
  return fetch(URL).then(res => res.ok ? res.json() : Promise.reject(res.status));
};

// NOTA: NOMINATIM pide User-Agent obligatorio, cache agresivo (1 req/seg)
const reverseGeocode = (lat, lon) => {
  const URL = `https://nominatim.openstreetmap.org/reverse?lat=${lat}&lon=${lon}&format=jsonv2&zoom=10&accept-language=es`;
  return fetch(URL, { headers: { 'User-Agent': 'Mears/1.0 (contact@mears.app)' } })
    .then(res => res.ok ? res.json() : Promise.reject(res.status));
};
```

4.4 Flujo Frontend – Backend

5. Seguridad en Base de Datos y APIs

5.1 Autenticación: JWT + bcrypt

Hashing de contraseñas

```
const SALT_ROUNDS = 12;
const HASH_FORMAT = /^$2[aby]$[0-9]{2}$[./A-Za-z0-9]{53}$/;
const isBcryptHash = (s) => typeof s === 'string' && HASH_FORMAT.test(s);
```

Sistema de tokens JWT dual

Token	Duración	Almacenamiento	Propósito
Access Token	15 minutos	JWT_SECRET	Autenticar cada request
Refresh Token	7 días	JWT_REFRESH_SECRET	Renovar access token sin login

5.2 Autorización por capas

6. Seguridad en Base de Datos

Mecanismo	Implementación	Protege contra
Helmet	app.use(helmet())	Clickjacking, XSS, MIME sniffing
CORS	app.use(cors({ origin: process.env.FRONTEND_URL, credentials: true }))	Requests cross-origin no autorizados
Rate Limiting	app.rate_limit({ windows: 15, max: 100 })	Brute force, DoS básico
Validación	express-validator	Inyección de datos malformados
Sanitización	trim(), escape()	XSS, espacios en blanco
Hashing	bcrypt, SALT_ROUNDS = 12	Rainbow tables, fuerza bruta
Tokens	JWT_SECRET, JWT_REFRESH_SECRET	Falsificación de sesiones
Errores	Mensajes genéricos en producción	Information leakage
Logs	Sin contraseñas ni tokens	Exposición de credenciales

7. Gestión segura de errores

```
app.error(err, req, res, next) {
  console.error('[ERROR]', err.stack);
  res.status(err.status || 500).send({
    error: {
      message: err.message || 'Error interno del servidor',
      stack: err.stack
    }
  });
};
```

8. Reconexión automática de sesión

```
// Interceptor de respuestas: si 401, intentar refrescar
axios.interceptors.response.use(
  (response) => response,
  (error) => {
    const original = error.config;
    if (error.response?.status === 401 && !original._retry) {
      original._retry = true;
      axios.post('/auth/refresh', {refreshToken: localStorage.getItem('refreshToken')})
        .then(r => { localStorage.setItem('accessToken', r.data.accessToken);
                     return axios(original); })
        .catch(() => { localStorage.clear(); window.location.reload(); });
    }
    return Promise.reject(error);
  );
```

Resumen de Criterios

Criterio	Peso	Justificación clave
1 Motor de BD	10	PostgreSQL justificado con comparativa vs 3 alternativas, ventajas específicas y limitaciones
2 Diseño de BD	15	12 tablas, normalización 3FN, 5 índices, 7 relaciones, 4 constraints CHECK y UNIQUE
3 Conexión BBDD	10	Pool ESO local con timeouts, SSL, config dual (localhost / producción), queries 100% parametrizadas
4 APIs	15	44 REST endpoints (CRUD completo), JWT + bcrypt, 3 APIs externas con fallbacks y cache
5 Seguridad	10	JWT dual (15min / 7d), bcrypt 12 rondas, 3 capas autorización, 7 cabeceras protegidas